This page intentionally left blank

Prepared exclusively for Zach

Index

A

Accessor function, 31
ACM, *see* Association for Computing Machinery
Active code generator, 104
Activity diagram, 150
Advanced C++ Programming Styles and Idioms, 265
Advanced Programming in the Unix Environment, 264
Aegis transaction-based configuration management, 246, 271
Agent, 76, 117, 297
Algorithm
    binary chop, 180
    choosing, 182
    combinatoric, 180
    divide-and-conquer, 180
    estimating, 177, 178
    linear, 177
    O() notation, 178, 181
    quicksort, 180
    runtime, 181
    sublinear, 177
Allocations, nesting, 131
Analysis Patterns, 264
Anonymity, 258
AOP, *see* Aspect-Oriented Programming
Architecture
    deployment, 156
    flexibility, 46
    prototyping, 55
    temporal decoupling, 152
Art of Computer Programming, 183
Artificial intelligence, marauding, 26
Aspect-Oriented Programming (AOP), 39, 273
Assertion, 113, 122, 175
    side effects, 124
    turning off, 123
Association for Computing Machinery (ACM), 262
    Communications of the ACM, 263
    SIGPLAN, 263
Assumptions, testing, 175
"`at`" command, 231
Audience, 21
    needs, 19
`auto_ptr`, 134
Automation, 230
    approval procedures, 235
    build, 88, 233
    compiling, 232
    `cron`, 231
    documentation, 251
    scripts, 234
    team, 229
    testing, 29, 238
    Web site generation, 235
`awk`, 99

B

Backus-Naur Form (BNF), 59n
Base class, 112
`bash` shell, 80, 82n
Bean, *see* Enterprise Java Beans (EJB)
Beck, Kent, 194, 258
Beowulf project, 268
"Big *O*" notation, 177
"Big picture", 8
Binary chop, 97, 180
Binary format, 73
    problems parsing, 75
`bison`, 59, 269
BIST, *see* Built-In Self Test
Blackboard system, 165
    partitioning, 168
    workflow, 169

Prepared exclusively for Zach

Blender example
contract for, 119, 289
regression test jig, 305
workflow, 151
BNF, *see* Backus-Naur Form (BNF)
Boiled frog, 8, 175, 225
Boundary condition, 173, 243
Brain, Marshall, 265
Branding, 226
Brant, John, 268
"Broken Window Theory", 5
vs. stone soup, 9
Brooks, Fred, 264
Browser, class, 187
Browser, refactoring, 187, 268
Bug, 90
failed contract as, 111
see also Debugging; Error
Build
automation, 88, 233
dependencies, 233
final, 234
nightly, 231
refactoring, 187
Built-In Self Test (BIST), 189
Business logic, 146
Business policy, 203

C

C language
assertions, 122
DBC, 114
duplication, 29
error handling, 121
error messages, 115
macros, 121
Object Pascal interface, 101
C++ language, 46
assertions, 122
`auto_ptr`, 134
books, 265
DBC, 114
decoupling, 142
DOC++, 251, 269
duplication, 29
error messages, 115
exceptions, 132
unit tests, 193
Caching, 31
Call, routine, 115, 173
Cascading Style Sheets (CSS), 253
Cat
blaming, 3
herding, 224
Schrödinger's, 47
Catalyzing change, 8
Cathedrals, xx
Cetus links, 265
Change, catalyzing, 8
Christiansen, Tom, 81
Class
assertions, 113
base, 112
coupling, 139, 142
coupling ratios, 242
encapsulating resource, 132
invariant, 110, 113
number of states, 245
resource allocation, 132
subclass, 112
wrapper, 132, 133, 135, 141
Class browser, 187
ClearCase, 271
Cockburn, Alistair, xxiii, 205, 264, 272
Code generator, 28, 102
active, 104
makefiles, 232
parsers, 105
passive, 103
Code profiler, 182
Code reviews, 33, 236
Coding
algorithm speed, 177
comments, 29, 249
coupled, 130
coverage analysis, 245
database schema, 104
defensive, 107
and documentation, 29, 248
estimating, 68
exceptions, 125
implementation, 173
iterative, 69
"lazy", 111
metrics, 242
modules, 138
multiple representations, 28
orthogonality, 34, 36, 40

Prepared exclusively for Zach

ownership, 258
prototypes, 55
server code, 196
"shy", 40, 138
specifications, 219
tracer bullets, 49–51
unit testing, 190, 192
see also Coupled code; Decoupled code; Metadata; Source code control system (SCCS)
Cohesion, 35
COM, *see* Component Object Model
Combinatorial explosion, 140, 167
Combinatoric algorithm, 180
Command shell, 77
`bash`, 80
Cygwin, 80
vs. GUI, 78
UWIN, 81
Windows, 80
Comment, 29, 249
avoiding duplication, 29
DBC, 113
parameters, 250
types of, 249
unnecessary, 250
see also Documentation
Common Object Request Broker (CORBA), 29, 39, 46
Event Service, 160
Communicating, 18
audience, 19, 21
duplication, 32
e-mail, 22
and formal methods, 221
presentation, 20
style, 20
teams, 225
users, 256
writing, 18
Communications of the ACM, 263
Comp.object FAQ, 272
Compiling, 232
compilers, 267
DBC, 113
warnings and debugging, 92
Component Object Model (COM), 55
Component-based systems, *see* Modular system
Concurrency, 150
design, 154
interfaces, 155
and Programming by Coincidence, 154
requirements analysis of, 150
workflow, 150
Concurrent Version System (CVS), 271
Configuration
cooperative, 148
dynamic, 144
metadata, 147
Configuration management, 86, 271
Constantine, Larry L., 35
Constraint management, 213
Constructor, 132
initialization, 155
Contact, authors' e-mail, xxiii
Context, use instead of globals, 40
Contract, 109, 174
see also Design by contract (DBC)
Controller (MVC), 162
Coplien, Jim, 265
CORBA, *see* Common Object Request Broker
Coupled code, 130
coupling ratios, 242
minimizing, 138, 158
performance, 142
temporal coupling, 150
see also Decoupled code
Coverage analysis, 245
Cox, Brad J., 189n
Crash, 120
Critical thinking, 16
`cron`, 231
CSS, *see* Cascading Style Sheets
CVS, *see* Concurrent Version System
Cygwin, 80, 270

D

Data
blackboard system, 169
caching, 31
dictionary, 144
dynamic data structures, 135
global, 40
language, 60
normalizing, 30

Prepared exclusively for Zach

readable *vs.* understandable, 75
test, 100, 243
views, 160
visualizing, 93
see also Metadata
Data Display Debugger (DDD), 93, 268
Database
active code generator, 104
schema, 105f, 141, 144
schema maintenance, 100
DBC, *see* Design by contract
DDD, *see* Data Display Debugger
Deadline, 6, 246
Deadlock, 131
Debugging, 90
assertions, 123
binary search, 97
bug location, 96
bug reproduction, 93
checklist, 98
compiler warnings and, 92
corrupt variables, 95
"Heisenbug", 124
rubber ducking, 95
and source code branching, 87
surprise bug, 97
and testing, 92, 195
time bomb, 192
tracing, 94
view, 164
visualizing data, 93
Decision making, 46
Decoupled code, 38, 40
architecture, 152
blackboard system, 166
Law of Demeter, 140
metadata, 145
minimizing coupling, 138
modular testing, 244
physical decoupling, 142
temporal coupling, 150
workflow, 150
see also Coupled code
Defensive coding, 107
Delegation, 304
Delphi, 55
see also Object Pascal
Demeter project, 274
Demeter, Law of, 140
Dependency, reducing, *see* Modular system; Orthogonality
Deployment, 156
Deployment descriptor, 148
Design
accessor functions, 31
concurrency, 154
context, 174
deployment, 156
design/methodology testing, 242
metadata, 145
orthogonality, 34, 37
physical, 142
refactoring, 186
using services, 154
Design by contract (DBC), 109, 155
and agents, 117
assertions, 113
class invariant, 110
as comments, 113
dynamic contracts, 117
iContract, 268
language support, 114
list insertion example, 110
pre- and postcondition, 110, 113, 114
predicates, 110
unit testing, 190
Design Patterns, 264
observer, 158
singleton, 41
strategy, 41
Destructor, 132
Detectives, 165
Development tree, 87
Development, iterative, 69
Divide-and-conquer algorithm, 180
DOC++ documentation generator, 251, 269
DocBook, 254
Documentation
automatic updating, 251
and code, 29, 248
comments, 29, 113, 249, 251
executable, 251
formats, 253
HTML, 101
hypertext, 210
internal/external, 248

Prepared exclusively for Zach

invariant, 117
mark-up languages, 254
orthogonality, 42
outline, 18
requirements, 204
technical writers, 252
word processors, 252, 254
writing specifications, 218
see also Comment; Web documentation
Dodo, 148
Domain, problem, 58, 66
Don't repeat yourself, *see* DRY principle
Downloading source code, *see* Example code
Dr. Dobbs Journal, 263
DRY principle, 27, 29, 42
see also Duplication
Duck, rubber, *see* Rubber duck
Dumpty, Humpty, xxii, 165
Duplication, 26
code generators avoid, 28
and code reviews, 33
design errors, 30
documentation and code, 29
DRY principle, 27, 29
interdeveloper, 32
in languages, 29
multiple representations, 28
teams, 226
under time pressure, 32
types of, 27
Dynamic configuration, 144
Dynamic data structure, 135
Dynamics of Software Development, 264

E

E-mail, 22
address for feedback, xxiii
Editor, 82
auto-indenting, 85
cursor movement, 84
features, 83
generating code, 103
how many to learn, 82
template, 84
types of, 266
Windows notepad, 84
Effective C++, 265
Eiffel, 109, 114, 267
EJB, *see* Enterprise Java Beans
`elvis` editor, 267
Emacs editor, 84, 266
`Viper` `vi` emulator, 267
Embedded mini-language, 62, 145
Embellishment, 11
Encapsulation, object, 127, 158
Eno, Brian, 205
Enterprise Java Beans (EJB), 39, 147
Entropy, 4
Error
DBC messages, 115
design, 30
domain-specific, 59
early crash, 120
log messages, 196
orthogonality, 41
testing, 240, 247
see also Exception
Error handler, 127
Estimating, 64
accuracy, 64
algorithms, 177, 178
iterative, 69
models, 66
problem domain, 66
project schedules, 68
records, 68
testing, 182
Eton College, xxi
Event, 157
Event channel, 160
Example code
add logging, 40
airline reservations, 164, 296
`assert` macro, 122
`auto_ptr` example, 134
bad resource balancing, 129, 130
downloading, xxiii
exception error handling, 125
good resource balancing, 131
JavaDoc example, 250
method chaining, 139
normalized class, 31
open password file, 126
open user file, 127

Prepared exclusively for Zach

resources and exceptions, 132, 133
side effect, 124
spaghetti error handling, 125
square root, 190
string parsing with `StringTokenizer`, 156
string parsing with `strtok`, 155
unnormalized class, 30
Example code by name
`AOP`, 40
`Misc.java`, 156
`assert`, 122
`bad_balance.c`, 129, 130
`balance.cc`, 134
`balance.c`, 131–133
`class Line`, 30, 31
`exception`, 125
`findPeak`, 250
`interface Flight`, 164, 296
`misc.c`, 155
`openpasswd.java`, 126
`openuserfile.java`, 127
`plotDate`, 139
`side_effect`, 124
`spaghetti`, 125
`sqrt`, 190
Exception, 121
effects of, 127
and error handlers, 127
missing files, 126
resource balancing, 132
when to use, 125
Excuses, 3
Executable document, 251
`expect`, 269
Expert, *see* Guru
Expiring asset, 12
eXtensible Style Language (XSL), 253
Extinction, 148
eXtreme Programming, 238n, 258, 272

F

Feature creep, 10
Feedback, e-mail address, xxiii
File
exception, 126
header, 29
implementation, 29
log, 196
makefile, 232
source, 103
Final build, 234
Fish, dangers of, 34
Flexibility, 46
Formal methods, 220, 221
Four Posts Puzzle, 213
Fowler, Martin, xxiii, 186, 273
Free Software Foundation, *see* GNU Project
Frog, boiled, *see* Boiled frog
Function
accessor, 31
Law of Demeter for ∼*s*, 140
similar, 41

G

Gamma, Erich, 194
Garbage collection, 134
Gardening metaphor, 184
Gehrke, Peter, xxiv
Glass, Robert, 221, 236
Global variables, 40, 130, 154
Glossary, project, 210
GNU Project, 274
C/C++ compiler, 267
General Public License (GPL), 80
GNU Image Manipulation Program (GIMP), 274
SmallEiffel, 267
"Good-enough software", *see* Software, quality
Gordian knot, 212
Goto, 127
GUI system
vs. command shell, 78
interface, 78
testing, 244
Guru, 17, 198

H

Hash, secure, 74
Header file, 29
"Heisenbug", 124, 289
Helicopter, 34n
Hopper, Grace, 8n, 90
"Hot-key" sequence, 196

Prepared exclusively for Zach

HTTP Web server, 196
Human factors, 241
Humpty Dumpty, xxii, 165
Hungarian notation, 249
Hungry consumer model, 153
Hypertext document, 210

I

iContract, 110, 114, 268
IDE, *see* Integrated Development Environment
IEEE Computer Society, 262
    IEEE Computer, 262
    IEEE Software, 263
Imperative language, 60
Implementation
    accidents, 173
    coding, 173
    specifications, 219
Imposed duplication, 28
Inadvertent duplication, 30
Indentation, automatic, 85
Independence, *see* Orthogonality
Infrastructure, 37
Inheritance, 111
    assertions, 113
    fan-in/fan-out, 242
Inner tennis, 215
Inspection, code, *see* Code reviews
Insure++, 136
Integrated circuit, 189n
Integrated Development Environment (IDE), 72, 232
Integration platform, 50
Integration testing, 239
Interface
    blackboard system, 168
    C/Object Pascal, 101
    concurrency, 155
    error handler, 128
    GUI, 78
    prototyping, 55
    user, 203
Invariant, 110, 113, 155
    loop, 116
    semantic, 116, 135
ISO9660 format, 233n
Iterative development, 69

J

Jacobson, Ivar, 204
Jargon, xxii, 210
Jargon file, 273
Java, 46, 267
    code generation, 232
    DBC, 114
    Enterprise Java Beans, 39, 147
    error messages, 115
    exceptions, 121
    iContract, 110, 114, 268
    `javaCC`, 59, 269
    JavaDoc, 248, 251
    JavaSpaces, 166, 273
    JUnit, 195
    multithreaded programming, 154
    property access, 100
    property files, 145
    resource balancing, 134
    RMI, 128
    string parser, 156
    tree view, 161
    unit tests, 193
    and Windows shells, 81
JavaDoc, *see* Java

K

K Desktop Environment, 273
Kaizen, xxi, 14
    see also Knowledge portfolio
Kernighan, Brian, 99
Keybinding, 82
Kirk, James T., 26
Knowledge
    producers and consumers, 166
Knowledge portfolio, 12
    building, 13
    critical thinking, 16
    learning and reading, 14
    researching, 15
Knuth, Donald, 183, 248
Korn, David, 81
Kramer, Reto, xxiv
Kruchten, Phillipe, 227n

L

Lakos, John, xxiv, 9, 142, 265
Lame excuses, 3

Prepared exclusively for Zach

Language, programming
conversions, 103, 105
DBC, 114
domain, 57
duplication in, 29
learning, 14
prototypes, 55
scripting, 55, 145
specification, 58, 62
text manipulation, 99
see also Mini-language
Large-Scale C++ Software Design, 142, 265
LaTeX system, 103
Law of Demeter, 140
Lawns, care of, xxi
Layered design, 37
Layered system, *see* Modular system
"lazy" code, 111
Lex and Yacc, 59
Librarian, *see* Project librarian
Library code, 39
Linda model, 167
Linear algorithms, 177
Linux, 15, 254, 265
Liskov Substitution Principle, 111
Listening, 21
Literate programming, 248
Logging, 39, 196
see also Tracing
Lookup table, 104
Loop
nested, 180
simple, 180
Loop invariant, 116

M

Macro, 78, 86
assertions, 122
documentation, 252
error handling, 121
Maintenance, 26
imperative languages, 61
Makefile, 232
recursive, 233
Managing expectations, 256
Mark-up language, 254
Martin, Robert C., 273
McCabe Cyclomatic Complexity Metric, 242
Member variables, *see* Accessor functions
Memory allocation, 135
Metadata, 144, 203
business logic, 146
configuration, 147
controlling transactions, 39
decoupled code, 145
and formal methods, 221
in plain text, 74
Metric, 242
Meyer, Bertrand, 31n, 109, 184, 264
Meyer, Scott, 265
Microsoft Visual C++, 198
Microsoft Windows, 46
Mini-language, 59
data language, 60
embedded, 62
imperative, 60
parsing, 62
stand-alone, 62
Mixing board, 205
MKS Source Integrity, 271
Model, 160
calculations, 67
components and parameters, 66
and estimating, 66
executable documents, 251
view, 162
Model-view-controller (MVC), 38, 160
Modular system, 37
coding, 138
prototyping, 55
resource allocation, 135
reversibility, 45
testing, 41, 190, 244
More Effective C++, 265
Mozilla, 273
Multithreaded programming, 154
MVC, *see* Model-view-controller
The Mythical Man Month, 264

N

Name, variable, 249
Nana, 114, 268
Nest allocations, 131
Nested loop, 180

Prepared exclusively for Zach

Netscape, 145, 273
Newsgroup, 15, 17, 33
Nonorthogonal system, 34
Normalize, 30
Novobilski, Andrew J., 189n

O

O() notation, 178, 181
Object
    coupling, 140n
    destruction, 133, 134
    persistence, 39
    publish/subscribe protocol, 158
    singleton, 41
    valid/invalid state, 154
    viewer, 163
Object Management Group (OMG), 270
Object Pascal, 29
    C interface, 101
Object-Oriented Programming, 189n
Object-Oriented Software Construction, 264
Obsolescence, 74
OLTP, *see* On-Line Transaction Processing system
OMG, *see* Object Management Group
On-Line Transaction Processing system (OLTP), 152
Options, providing, 3
Ordering, *see* Workflow
Orthogonality, 34
    coding, 34, 36, 40
    design, 37
    documentation, 42
    DRY principle, 42
    nonorthogonal system, 34
    productivity, 35
    project teams, 36, 227
    testing, 41
    toolkits & libraries, 39
    see also Modular system
Over embellishment, 11

P

Pain management, 185
`paint()` method, 173
Painting, 11
Papua New Guinea, 16
Parallel programming, 150
Parrots, killer, *see* Branding
Parsing, 59
    code generators, 105
    log messages, 196
    mini-language, 62
    strings, 155
Partitioning, 168
Pascal, 29
Passive code generator, 103
Performance testing, 241
Perl, 55, 62, 99
    C/Object Pascal interface, 101
    database schema generation, 100
    home page, 267
    Java property access, 100
    power tools, 270
    test data generation, 100
    testing, 197
    and typesetting, 100
    Unix utilities in, 81
    web documentation, 101
Perl Journal, 263
Persistence, 39, 45
Petzold, Charles, 265
Pike, Rob, 99
Pilot
    landing, handling, etc., 217
    who ate fish, 34
Plain text, 73
    vs. binary format, 73
    drawbacks, 74
    executable documents, 251
    leverage, 75
    obsolescence, 74
    and easier testing, 76
    Unix, 76
Polymorphism, 111
Post-it note, 53, 55
Powerbuilder, 55
The Practice of Programming, 99
Pragmatic programmer
    characteristics, xviii
    e-mail address, xxiii
    Web site, xxiii
Pre- and postcondition, 110, 113, 114
Predicate logic, 110
Preprocessor, 114
Presentation, 20

Prepared exclusively for Zach

Problem domain, 58, 66
metadata, 146
Problem solving, 213
checklist for, 214
Productivity, 10, 35
Programming by coincidence, 173
Programming staff
expense of, 237
Programming Windows, 265
Project
glossary, 210
"heads", 228
saboteur, 244
schedules, 68
see also Automation; Team, project
Project librarian, 33, 226
Prototyping, 53, 216
architecture, 55
disposable code, 56
kinds of, 54
and programming languages, 55
and tracer code, 51
using, 54
Publish/subscribe protocol, 158
Pugh, Greg, 95n
Purify, 136
PVCS Configuration Management, 271
Python, 55, 99, 267

Q

Quality
control, 9
requirements, 11
teams, 225
Quarry worker's creed, xx
Quicksort algorithm, 180

R

Rational Unified Process, 227n
Raymond, Eric S., 273
RCS, *see* Revision Control System
Real-world data, 243
Refactoring, 5, 185
automatic, 187
and design, 186
testing, 187
time constraints, 185
Refactoring browser, 187, 268
Refinement, excessive, 11
Regression, 76, 197, 232, 242
Relationship
has-a, 304
kind-of, 111, 304
Releases, and SCCS, 87
Remote Method Invocation (RMI), 128
exception handling, 39
Remote procedure call (RPC), 29, 39
Repository, 87
Requirement, 11, 202
business problem, 203
changing, 26
creep, 209
DBC, 110
distribution, 211
documenting, 204
in domain language, 58
expressing as invariant, 116
formal methods, 220
glossary, 210
over specifying, 208
and policy, 203
usability testing, 241
user interface, 203
Researching, 15
Resource balancing, 129
C++ exceptions, 132
checking, 135
coupled code, 130
dynamic data structures, 135
encapsulation in class, 132
Java, 134
nest allocations, 131
Response set, 141, 242
Responsibility, 2, 250, 258
Reuse, 33, 36
Reversibility, 44
flexible architecture, 46
Revision Control System (RCS), 250, 271
Risk management, 13
orthogonality, 36
RMI, *see* Remote Method Invocation
Rock-n-roll, 47
RPC, *see* Remote procedure call
Rubber ducking, 3, 95
Rules engine, 169

Prepared exclusively for Zach

S

Saboteur, 244
Samba, 272
Sample programs, *see* Example code
Sather, 114, 268
SCCS, *see* Source code control system
Schedule, project, 68
Schrödinger, Erwin (and his cat), 47
Scope, requirement, 209
Screen scraping, 61
Scripting language, 55, 145
Secure hash, 74
`sed`, 99
Sedgewick, Robert, 183
Self-contained components, *see* Orthogonality; Cohesion
Semantic invariant, 116, 135
`sendmail` program, 60
Sequence diagram, 158
Server code, 196
Services, design using, 154
Shell, command, 77
  vs. GUI, 78
  see also Command shell
"Shy code", 40
Side effect, 124
SIGPLAN, 263
Simple loop, 180
Singleton object, 41
Slashdot, 265
SmallEiffel, 267
Smalltalk, 46, 186, 187, 268, 272
Software
  development technologies, 221
  quality, 9
  requirements, 11
Software bus, 159
"Software Construction", 184
Software Development Magazine, 263
Software IC, 189n
"Software rot", 4
Solaris, 76
Source code
  cat eating, 3
  documentation, *see* Comments
  downloading, *see* Example code
  duplication in, 29
  generating, 103
  reviews, *see* Code reviews
Source code control system (SCCS), 86
  Aegis, 246
  builds using, 88
  CVS, 271
  development tree, 87
  plain text and, 76
  RCS, 250, 271
  repository, 87
  tools, 271
Specialization, 221
Specification, 58
  implementation, 219
  language, 62
  as security blanket, 219
  writing, 218
Spy cells, 138
Squeak, 268
Stand-alone mini-language, 62
"Start-up fatigue", 7
Starting a project
  problem solving, 212
  prototyping, 216
  specifications, 217
  see also Requirement
Stevens, W. Richard, 264
Stone soup, 7
  vs. broken windows, 9
Stone-cutter's creed, xx
String parser, 155
Stroop effect, 249
`strtok` routine, 155
Structured walkthroughs, *see* Code reviews
Style sheet, 20, 254
Style, communication, 20
Subclass, 112
Sublinear algorithm, 177
Supplier, *see* Vendor
Surviving Object-Oriented Projects: A Manager's Guide, 264
SWIG, 55, 270
Synchronization bar, 151
Syntax highlighting, 84
Synthetic data, 243

T

T Spaces, 166, 269
TAM, *see* Test Access Mechanism
Tcl, 55, 99, 269

Prepared exclusively for Zach

Team, project, 36, 224
automation, 229
avoiding duplication, 32
code review, 236
communication, 225
duplication, 226
functionality, 227
organization, 227
pragmatism in, xx
quality, 225
tool builders, 229
Technical writer, 252
Template, use case, 205
Temporal coupling, 150
Test Access Mechanism (TAM), 189
Test harness, 194
Testing
automated, 238
from specification, 29
bug fixing, 247
coverage analysis, 245
and culture, 197
debugging, 92, 196
design/methodology, 242
effectiveness, 244
estimates, 182
frequency, 246
GUI systems, 244
integration, 239
orthogonality, 36, 41
performance, 241
role of plain text, 76
refactoring, 187
regression, 76, 197, 232, 242
resource exhaustion, 240
saboteur, 244
test data, 100, 243
usability, 241
validation and verification, 239
see also Unit testing
Text manipulation language, 99
TOM programming language, 268
Toolkits, 39
Tools, adaptable, 205
Tracer code, 49
advantages of, 50
and prototyping, 51
Tracing, 94
see also Logging
Trade paper, 263
Trade-offs, 249
Transactions, EJB, 39
Tree widget, 161
troff system, 103
Tuple space, 167

U

UML, *see* Unified modeling language (UML)
UNDO key, 86
Unified modeling language (UML)
activity diagram, 150
sequence diagram, 158
use case diagram, 208
Uniform Access Principle, 31n
Unit testing, 190
DBC, 190
modules, 239
test harness, 194
test window, 196
writing tests, 193
Unix, 46, 76
Application Default files, 145
books, 264
Cygwin, 270
DOS tools, 270
Samba, 272
UWIN, 81, 270
Unix Network Programming, 264
Usability testing, 241
Use case, 204
diagrams, 206
Usenet newsgroup, 15, 17, 33
User
expectations, 256
groups, 18
interface, 203
requirements, 10
UWIN, 81, 270

V

Variable
corrupt, 95
global, 130, 154
name, 249
Vendor
libraries, 39
reducing reliance on, 36, 39, 46

Prepared exclusively for Zach

`vi` editor, 266
View
    debugging, 164
    executable documents, 251
    Java tree view, 161
    model-view-controller, 160, 162
    model-viewer network, 162
`vim` editor, 266
Visual Basic, 55
Visual C++, 198
Visual SourceSafe, 271
VisualWorks, 268

W

Walkthoughs, *see* Code reviews
Warnings, compilation, 92
Web documentation, 101, 210, 253
    automatic generation, 235
    news and information, 265
Web server, 196
Web site, pragmatic programmer, xxiii
What You See Is What You Get (WYSIWYG), 78
WikiWikiWeb, 265
Win32 System Services, 265
Windows, 46
    "`at`" command, 231
    books, 265
    Cygwin, 80
    metadata, 145
    notepad, 84
    Unix utilities, 80, 81
    UWIN, 81
WinZip, 272
WISDOM acrostic, 20
Wizard, 198
Word processor, 252, 254
Workflow, 150
    blackboard system, 169
    content-driven, 234
Wrapper, 132, 133, 135, 141
Writing, 18
    see also Documentation
`www.pragmaticprogrammer.com`, xxiii
WYSIWYG, *see* What You See Is What You Get

X

XEmacs editor, 266
Xerox Parc, 39
XSL, *see* eXtensible Style Language
xUnit, 194, 269

Y

`yacc`, 59
Yourdon, Edward, 10, 35
Y2K problem, 32, 208

Z

Z shell, 272

Prepared exclusively for Zach

informIT.com THE TRUSTED TECHNOLOGY LEARNING SOURCE

PEARSON

InformIT is a brand of Pearson and the online presence for the world's leading technology publishers. It's your source for reliable and qualified content and knowledge, providing access to the top brands, authors, and contributors from the tech community.

Addison-Wesley Cisco Press EXAM/CRAM IBM Press QUE PRENTICE HALL SAMS | Safari Books Online

LearnIT at InformIT

Looking for a book, eBook, or training video on a new technology? Seeking timely and relevant information and tutorials? Looking for expert opinions, advice, and tips? **InformIT has the solution.**

- Learn about new releases and special promotions by subscribing to a wide variety of newsletters. Visit **informit.com/newsletters**.
- Access FREE podcasts from experts at **informit.com/podcasts**.
- Read the latest author articles and sample chapters at **informit.com/articles**.
- Access thousands of books and videos in the Safari Books Online digital library at **safari.informit.com**.
- Get tips from expert blogs at **informit.com/blogs**.

Visit **informit.com/learn** to discover all the ways you can access the hottest technology content.

Are You Part of the IT Crowd?

Connect with Pearson authors and editors via RSS feeds, Facebook, Twitter, YouTube, and more! Visit **informit.com/socialconnect**.

informIT.com THE TRUSTED TECHNOLOGY LEARNING SOURCE PEARSON

Addison-Wesley Cisco Press EXAM/CRAM IBM Press QUE PRENTICE HALL SAMS | Safari Books Online

Prepared exclusively for Zach

Addison Wesley

THIS PRODUCT

informit.com/register

Register the Addison-Wesley, Exam Cram, Prentice Hall, Que, and Sams products you own to unlock great benefits.

To begin the registration process, simply go to **informit.com/register** to sign in or create an account. You will then be prompted to enter the 10- or 13-digit ISBN that appears on the back cover of your product.

Registering your products can unlock the following benefits:

- Access to supplemental content, including bonus chapters, source code, or project files.
- A coupon to be used on your next purchase.

Registration benefits vary by product. Benefits will be listed on your Account page under Registered Products.

About InformIT — THE TRUSTED TECHNOLOGY LEARNING SOURCE

INFORMIT IS HOME TO THE LEADING TECHNOLOGY PUBLISHING IMPRINTS Addison-Wesley Professional, Cisco Press, Exam Cram, IBM Press, Prentice Hall Professional, Que, and Sams. Here you will gain access to quality and trusted content and resources from the authors, creators, innovators, and leaders of technology. Whether you're looking for a book on a new technology, a helpful article, timely newsletters, or access to the Safari Books Online digital library, InformIT has a solution for you.

informIT.com
THE TRUSTED TECHNOLOGY LEARNING SOURCE

Addison-Wesley | Cisco Press | Exam Cram
IBM Press | Que | Prentice Hall | Sams

SAFARI BOOKS ONLINE

Prepared exclusively for Zach

The Pragmatic Programmer

This card summarizes the tips and checklists found in *The Pragmatic Programmer.*

For more information about THE PRAGMATIC PROGRAMMERS LLC, source code for the examples, up-to-date pointers to Web resources, and an online bibliography, visit us at `www.pragmaticprogrammer.com`.

Quick Reference Guide

TIPS 1 TO 22

1. Care About Your Craft xix
Why spend your life developing software unless you care about doing it well?

2. Think! About Your Work xix
Turn off the autopilot and take control. Constantly critique and appraise your work.

3. Provide Options, Don't Make Lame Excuses 3
Instead of excuses, provide options. Don't say it can't be done; explain what *can* be done.

4. Don't Live with Broken Windows 5
Fix bad designs, wrong decisions, and poor code when you see them.

5. Be a Catalyst for Change 8
You can't force change on people. Instead, show them how the future might be and help them participate in creating it.

6. Remember the Big Picture 8
Don't get so engrossed in the details that you forget to check what's happening around you.

7. Make Quality a Requirements Issue 11
Involve your users in determining the project's real quality requirements.

8. Invest Regularly in Your Knowledge Portfolio 14
Make learning a habit.

9. Critically Analyze What You Read and Hear 16
Don't be swayed by vendors, media hype, or dogma. Analyze information in terms of you and your project.

10. It's Both What You Say and the Way You Say It .. 21
There's no point in having great ideas if you don't communicate them effectively.

11. *DRY*—Don't Repeat Yourself 27
Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.

12. Make It Easy to Reuse 33
If it's easy to reuse, people will. Create an environment that supports reuse.

13. Eliminate Effects Between Unrelated Things 35
Design components that are self-contained, independent, and have a single, well-defined purpose.

14. There Are No Final Decisions 46
No decision is cast in stone. Instead, consider each as being written in the sand at the beach, and plan for change.

15. Use Tracer Bullets to Find the Target 49
Tracer bullets let you home in on your target by trying things and seeing how close they land.

16. Prototype to Learn 54
Prototyping is a learning experience. Its value lies not in the code you produce, but in the lessons you learn.

17. Program Close to the Problem Domain 58
Design and code in your user's language.

18. Estimate to Avoid Surprises 64
Estimate before you start. You'll spot potential problems up front.

19. Iterate the Schedule with the Code 69
Use experience you gain as you implement to refine the project time scales.

20. Keep Knowledge in Plain Text 74
Plain text won't become obsolete. It helps leverage your work and simplifies debugging and testing.

21. Use the Power of Command Shells 80
Use the shell when graphical user interfaces don't cut it.

22. Use a Single Editor Well 82
The editor should be an extension of your hand; make sure your editor is configurable, extensible, and programmable.

Prepared exclusively for Zach

23. Always Use Source Code Control 88
Source code control is a time machine for your work—you *can* go back.

24. Fix the Problem, Not the Blame 91
It doesn't really matter whether the bug is your fault or someone else's—it is still your problem, and it still needs to be fixed.

25. Don't Panic When Debugging 91
Take a deep breath and THINK! about what could be causing the bug.

26. "select" Isn't Broken 96
It is rare to find a bug in the OS or the compiler, or even a third-party product or library. The bug is most likely in the application.

27. Don't Assume It—Prove It 97
Prove your assumptions in the actual environment—with real data and boundary conditions.

28. Learn a Text Manipulation Language 100
You spend a large part of each day working with text. Why not have the computer do some of it for you?

29. Write Code That Writes Code 103
Code generators increase your productivity and help avoid duplication.

30. You Can't Write Perfect Software 107
Software can't be perfect. Protect your code and users from the inevitable errors.

31. Design with Contracts 111
Use contracts to document and verify that code does no more and no less than it claims to do.

32. Crash Early 120
A dead program normally does a lot less damage than a crippled one.

33. Use Assertions to Prevent the Impossible 122
Assertions validate your assumptions. Use them to protect your code from an uncertain world.

34. Use Exceptions for Exceptional Problems 127
Exceptions can suffer from all the readability and maintainability problems of classic spaghetti code. Reserve exceptions for exceptional things.

35. Finish What You Start 129
Where possible, the routine or object that allocates a resource should be responsible for deallocating it.

36. Minimize Coupling Between Modules 140
Avoid coupling by writing "shy" code and applying the Law of Demeter.

37. Configure, Don't Integrate 144
Implement technology choices for an application as configuration options, not through integration or engineering.

38. Put Abstractions in Code, Details in Metadata . 145
Program for the general case, and put the specifics outside the compiled code base.

39. Analyze Workflow to Improve Concurrency 151
Exploit concurrency in your user's workflow.

40. Design Using Services 154
Design in terms of *services*—independent, concurrent objects behind well-defined, consistent interfaces.

41. Always Design for Concurrency 156
Allow for concurrency, and you'll design cleaner interfaces with fewer assumptions.

42. Separate Views from Models 161
Gain flexibility at low cost by designing your application in terms of models and views.

43. Use Blackboards to Coordinate Workflow 169
Use blackboards to coordinate disparate facts and agents, while maintaining independence and isolation among participants.

44. Don't Program by Coincidence 175
Rely only on reliable things. Beware of accidental complexity, and don't confuse a happy coincidence with a purposeful plan.

45. Estimate the Order of Your Algorithms 181
Get a feel for how long things are likely to take *before* you write code.

46. Test Your Estimates 182
Mathematical analysis of algorithms doesn't tell you everything. Try timing your code in its target environment.

Prepared exclusively for Zach

47. Refactor Early, Refactor Often 186
Just as you might weed and rearrange a garden, rewrite, rework, and re-architect code when it needs it. Fix the root of the problem.

48. Design to Test 192
Start thinking about testing before you write a line of code.

49. Test Your Software, or Your Users Will 197
Test ruthlessly. Don't make your users find bugs for you.

50. Don't Use Wizard Code You Don't Understand . 199
Wizards can generate reams of code. Make sure you understand *all* of it before you incorporate it into your project.

51. Don't Gather Requirements—Dig for Them 202
Requirements rarely lie on the surface. They're buried deep beneath layers of assumptions, misconceptions, and politics.

52. Work with a User to Think Like a User 204
It's the best way to gain insight into how the system will *really* be used.

53. Abstractions Live Longer than Details 209
Invest in the abstraction, not the implementation. Abstractions can survive the barrage of changes from different implementations and new technologies.

54. Use a Project Glossary 210
Create and maintain a single source of all the specific terms and vocabulary for a project.

55. Don't Think Outside the Box—*Find* the Box 213
When faced with an impossible problem, identify the *real* constraints. Ask yourself: "Does it have to be done this way? Does it have to be done at all?"

56. Start When You're Ready 215
You've been building experience all your life. Don't ignore niggling doubts.

57. Some Things Are Better Done than Described . 218
Don't fall into the specification spiral—at some point you need to start coding.

58. Don't Be a Slave to Formal Methods 220
Don't blindly adopt any technique without putting it into the context of your development practices and capabilities.

59. Costly Tools Don't Produce Better Designs 222
Beware of vendor hype, industry dogma, and the aura of the price tag. Judge tools on their merits.

60. Organize Teams Around Functionality 227
Don't separate designers from coders, testers from data modelers. Build teams the way you build code.

61. Don't Use Manual Procedures 231
A shell script or batch file will execute the same instructions, in the same order, time after time.

62. Test Early. Test Often. Test Automatically. 237
Tests that run with every build are much more effective than test plans that sit on a shelf.

63. Coding Ain't Done 'Til All the Tests Run 238
'Nuff said.

64. Use Saboteurs to Test Your Testing 244
Introduce bugs on purpose in a separate copy of the source to verify that testing will catch them.

65. Test State Coverage, Not Code Coverage 245
Identify and test significant program states. Just testing lines of code isn't enough.

66. Find Bugs Once 247
Once a human tester finds a bug, it should be the *last* time a human tester finds that bug. Automatic tests should check for it from then on.

67. English is Just a Programming Language 248
Write documents as you would write code: honor the *DRY* principle, use metadata, MVC, automatic generation, and so on.

68. Build Documentation In, Don't Bolt It On 248
Documentation created separately from code is less likely to be correct and up to date.

69. Gently Exceed Your Users' Expectations 255
Come to understand your users' expectations, then deliver just that little bit more.

70. Sign Your Work 258
Craftsmen of an earlier age were proud to sign their work. You should be, too.

Prepared exclusively for Zach

Checklists

✔ Languages to Learn ..... page 17

Tired of C, C++, and Java? Try CLOS, Dylan, Eiffel, Objective C, Prolog, Smalltalk, or TOM. Each of these languages has different capabilities and a different "flavor." Try a small project at home using one or more of them.

✔ The WISDOM Acrostic ..... page 20

What do you want them to learn?
What is their **i**nterest in what you've got to say?
How **s**ophisticated are they?
How much **d**etail do they want?
Whom do you want to **o**wn the information?
How can you **m**otivate them to listen to you?

✔ How to Maintain Orthogonality ..... page 34

- Design independent, well-defined components.
- Keep your code decoupled.
- Avoid global data.
- Refactor similar functions.

✔ Things to prototype ..... page 53

- Architecture
- New functionality in an existing system
- Structure or contents of external data
- Third-party tools or components
- Performance issues
- User interface design

✔ Architectural Questions ..... page 55

- Are responsibilities well defined?
- Are the collaborations well defined?
- Is coupling minimized?
- Can you identify potential duplication?
- Are interface definitions and constraints acceptable?
- Can modules access needed data—*when* needed?

✔ Debugging Checklist ..... page 98

- Is the problem being reported a direct result of the underlying bug, or merely a symptom?
- Is the bug *really* in the compiler? Is it in the OS? Or is it in your code?
- If you explained this problem in detail to a coworker, what would you say?
- If the suspect code passes its unit tests, are the tests complete enough? What happens if you run the unit test with *this* data?
- Do the conditions that caused this bug exist anywhere else in the system?

✔ Law of Demeter for Functions ..... page 141

An object's method should call only methods belonging to:

- Itself
- Any parameters passed in
- Objects it creates
- Component objects

✔ How to Program Deliberately ..... page 172

- Stay aware of what you're doing.
- Don't code blindfolded.
- Proceed from a plan.
- Rely only on reliable things.
- Document your assumptions.
- Test assumptions as well as code.
- Prioritize your effort.
- Don't be a slave to history.

✔ When to Refactor ..... page 185

- You discover a violation of the *DRY* principle.
- You find things that could be more orthogonal.
- Your knowledge improves.
- The requirements evolve.
- You need to improve performance.

✔ Cutting the Gordian Knot ..... page 212

When solving *impossible* problems, ask yourself:

- Is there an easier way?
- Am I solving the right problem?
- *Why* is this a problem?
- What makes it hard?
- Do I have to do it this way?
- Does it have to be done at all?

✔ Aspects of Testing ..... page 237

- Unit testing
- Integration testing
- Validation and verification
- Resource exhaustion, errors, and recovery
- Performance testing
- Usability testing
- Testing the tests themselves

Checklists from *The Pragmatic Programmer*, by Andrew Hunt and David Thomas. Visit `www.pragmaticprogrammer.com`.

Copyright © 2000 by Addison Wesley Longman, Inc.

Prepared exclusively for Zach